Q) PEAS

Problem statement relevant to Artificial Intelligence:
Indoor delivery robot.
1. Problem Identification:
An indoor delivery robot is designed to deliver the packages, food or document inside offices, campuses or hotels while navigating dynamic environment.
The robot must:
i) Navigate automously inside building.
ii) Detect and avoid obstacles using AI and sensors.
iii) Identify the correct destination / location.
iv) Carry and securely deliver items.
v) Plan the shortest and safest route.
vi) Notify the users when delivery is completed.
2. PEAS description:
i) Performance Measure (P): Defines how the Agent's success is evaluated.
Accurate delivery
Minimum delivery time
Obstacle avoidance
Safe navigation
Low energy consumption.
ii) Environment (E): The surrounding in which the agent operates.
Hospital, offices, colleges, hotels, corridors, rooms, elevators, indoor pathways.
iii) Actuators (A): The component that aagent uses to perform actions.
Motors, robotic arm or delivery compartment, buzzer, display / speakers.
iv) Sensors (S): The devices that agent uses to perceive and collect information from its environment.
Camera, LIDAR, ultrasonic sensor, IR sensors, GPS, IMU, wheel encoders.
3. Problem formulation:
i) Initial State:
Robot is at the charging station or starting point with delivery item.
ii) State space:
All possible locations and path inside the building.
iii) Actions:
Move forward, backward, turn left / right, avoid obstacles, pick up item, deliver item, stop.
iv) Transition model:
The robot changes its location based on selected action while considering obstacles and environment.
v) Goal state:
The robot successfully reaches the destination and delivers the item safely.
vi) Path Cost:
Minimize travel time, distance, energy consumption and avoid collision.
Conclusion:
The AI-based indoor delivery robot improves efficiency by providing fast, safe and autonomous deliveries.

Q) Write and Execute basic programs in **PROLOG**

1) Student Grade

```
grade(Marks, 'A') :-
        Marks >= 75.
grade(Marks, 'B') :-
        Marks >= 60,
        Marks < 75.
grade(Marks, 'C') :-
        Marks >= 35,
        Marks < 60.
grade(Marks, 'Fail') :-
        Marks <35.
```

- ***grade*(78, G).**

2)Checking voting elgibility

```
vote(Age, 'Eligible') :-
        Age >= 18.
vote(Age, 'Not Eligible') :-
        Age < 18.
```

- *vote*(20, G).

- *vote*(15, G).

3)Finding min/max

```
find_max(X, Y, X) :- X >= Y, !.
find_max(X, Y, Y) :- X < Y.
find_min(X, Y, X) :- X =< Y, !.
find_min(X, Y, Y) :- X > Y.
```

***-find_max*(100,200,Max).**

***-find_min*(100,200,Min).**

4)Find area

area_rectangle(Length, Breadth, Area) :- Area is Length * Breadth.

-*area_rectangle*(5, 8, Area).

5)Factorial

factorial(0, 1).

factorial(N, Result) :- N > 0,

N_minus_1 is N - 1, factorial(N_minus_1,

SubResult), Result is N * SubResult.

-*factorial*(5, F).

6)Celsius to Fahrenheit

celsius_to_fahrenheit(C, F) :- F is C * 9 / 5 + 32.

-*celsius_to_fahrenheit*(25, F).

7)Checking leap year

leap(Year) :-

    0 is Year mod 400.

leap(Year) :-

    0 is Year mod 4,

    Year mod 100 =\= 0.

- *leap*(2024).

8) Fact and rules

% Facts

likes(john, pizza).

likes(mary, pizza).

likes(rahul, burger).

% Rule

friends(X, Y) :-

likes(X, pizza),

likes(Y, pizza),

X \= Y.

***-friends*(john, mary).**

Q) DFS

```
def dfs(graph, start, goal):
    stack = [[start]]
    visited = set()
    while stack:
        path = stack.pop()
        node = path[-1]
        if node == goal:
            return path
        if node not in visited:
            visited.add(node)
            for neighbor in reversed(graph.get(node, [])):
                if neighbor not in visited:
                    stack.append(path + [neighbor])
    return None
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
```

```
}
start_node_dfs = 'A'
goal_node_dfs = 'F'
path_dfs = dfs(graph, start_node_dfs, goal_node_dfs)
if path_dfs:
    print(f"Path from {start_node_dfs} to {goal_node_dfs} (DFS): {path_dfs}")
else:
    print(f"No path found from {start_node_dfs} to {goal_node_dfs} (DFS).")
```

Q) BFS

```
from collections import deque
def bfs(graph, start, goal):
    queue = deque([[start]])
    visited = set([start])

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        for neighbor in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])

    return None
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
```

```
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}
start_node = 'A'
goal_node = 'F'
path = bfs(graph, start_node, goal_node)
if path:
    print(f"Shortest path from {start_node} to {goal_node}: {path}")
else:
    print(f"No path found from {start_node} to {goal_node}.")
```

Q) A* on on a graph

```
def astar(start, goal):
    open_set = {start}
    closed_set = set()
    g = {start: 0}
    parents = {start: None}

    while open_set:
        current = min(open_set, key=lambda x: g[x] + heuristic(x))

        if current == goal:
            path = []
            while current:
                path.append(current)
                current = parents[current]
            path.reverse()
            print("Shortest path:", path)
            print("Total cost:", g[goal])
            return
```

```
        open_set.remove(current)
        closed_set.add(current)

        neighbors = graph.get(current, [])
        for neighbor, cost in neighbors:
            if neighbor in closed_set:
                continue

            new_cost = g[current] + cost
            if neighbor not in open_set or new_cost < g.get(neighbor, float('inf')):
                g[neighbor] = new_cost
                parents[neighbor] = current
                open_set.add(neighbor)

    print("No path found.")
def heuristic(node):
    h = {
        'A': 11,
        'B': 6,
        'C': 99,
        'D': 1,
        'E': 7,
        'G': 0,
    }
    return h.get(node, float('inf'))
graph = {
    'A': [('B', 2), ('E', 3)],
    'B': [('C', 1), ('G', 9)],
    'C': [],
    'E': [('D', 6)],
    'D': [('G', 1)],
}
astar('A', 'G')
```

Q) A* for 8 puzzle

```
def h(state, goal):
    return sum(abs(i // 3 - goal.index(v) // 3) +
               abs(i % 3 - goal.index(v) % 3)
               for i, v in enumerate(state) if v != 0)
def get_neighbors(state):
    neighbors = []
    zero = state.index(0)
    r, c = zero // 3, zero % 3

    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            new = state[:]
            j = nr * 3 + nc
            new[zero], new[j] = new[j], new[zero]
            neighbors.append(new)

    return neighbors
def astar(start, goal):
    open_list = [(h(start, goal), 0, start, [start])]
    visited = set()

    while open_list:
        open_list.sort()
        f, g, state, path = open_list.pop(0)

        if state == goal:
            return path

        key = tuple(state)
        if key in visited:
            continue
        visited.add(key)
```

```
        for neighbor in get_neighbors(state):
            if tuple(neighbor) not in visited:
                ng = g + 1
                nf = ng + h(neighbor, goal)
                open_list.append((nf, ng, neighbor, path + [neighbor]))

    return None
start = [1, 2, 3, 4, 0, 6, 7, 5, 8]
goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]
path = astar(start, goal)
if path:
    for step in path:
        print(step[:3])
        print(step[3:6])
        print(step[6:9])
        print()
else:
    print("No solution found.")
```

Q) MINIMAX

```
import math
def minimax(depth, node_index, is_max, scores, height):
    if depth == height:
        return scores[node_index]

    if is_max:
        return max(
            minimax(depth + 1, node_index * 2, False, scores, height),
            minimax(depth + 1, node_index * 2 + 1, False, scores, height)
        )
    else:
        return min(
            minimax(depth + 1, node_index * 2, True, scores, height),
            minimax(depth + 1, node_index * 2 + 1, True, scores, height)
```

```
        )

scores = [3, 5, 6, 9, 1, 2, 0, -1]
height = int(math.log2(len(scores)))
best_score = minimax(0, 0, True, scores, height)
print("The optimal value is:", best_score)
```

Q) Neural network in C

```
#include <stdio.h>
int main()
{
        int input[4][2] = {
        {0, 0},
        {0, 1},
        {1, 0},
        {1, 1}
    };

    int target[4] = {0, 0, 0, 1};
    float w1 = 0.0;
    float w2 = 0.0;
    float bias = 0.0;

    float learning_rate = 0.1;

    int epochs = 10;
    for (int epoch = 0; epoch < epochs; epoch++)
    {
        printf("Epoch %d\n", epoch + 1);
```

```
    for (int i = 0; i < 4; i++)
    {
        float sum = (input[i][0] * w1) +
                    (input[i][1] * w2) +
                    bias;
        int output;

        if (sum >= 1.0)
            output = 1;
        else
            output = 0;
        int error = target[i] - output;
        w1 = w1 + learning_rate * error * input[i][0];

        w2 = w2 + learning_rate * error * input[i][1];

        bias = bias + learning_rate * error;

        printf("Input: %d %d | Target: %d | Output: %d | Error: %d\n",
                input[i][0],
                input[i][1],
                target[i],
                output,
                error);

        printf("\n");
    }

    printf("w1 = %.2f\n", w1);
    printf("w2 = %.2f\n", w2);
    printf("bias = %.2f\n", bias);
```

```
    }

    printf("Final AND Gate Results:\n");

    for (int i = 0; i < 4; i++)
    {
        float sum = (input[i][0] * w1) +
                    (input[i][1] * w2) +
                    bias;

        int output;

        if (sum >= 1.0)
            output = 1;
        else
            output = 0;

        printf("%d AND %d = %d\n",
               input[i][0],
               input[i][1],
               output);
    }

    printf("\nFinal weights:\n");
    printf("w1 = %.2f\n", w1);
    printf("w2 = %.2f\n", w2);
    printf("bias = %.2f\n", bias);

    return 0;
}
```

Q) Perceptron to lean AND,OR,XOR

```
import numpy as np
def train_perceptron(X, y, epochs=10, learning_rate=0.1):
    weights = np.zeros(X.shape[1])
    bias = 0

    for epoch in range(epochs):
        for i in range(len(X)):
            z = np.dot(X[i], weights) + bias
            prediction = 1 if z >= 0 else 0
            error = y[i] - prediction
            weights = weights + learning_rate * error * X[i]
            bias = bias + learning_rate * error

    return weights, bias

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

AND = np.array([0, 0, 0, 1])

OR = np.array([0, 1, 1, 1])

XOR = np.array([0, 1, 1, 0])
```

```
def test_perceptron(X, weights, bias):
    predictions = []

    for x in X:
        z = np.dot(x, weights) + bias
        prediction = 1 if z >= 0 else 0
        predictions.append(prediction)

    return predictions

w_and, b_and = train_perceptron(X, AND)
print("AND Gate")
print("Weights:", w_and)
print("Bias:", b_and)
print("Output:", test_perceptron(X, w_and, b_and))

w_or, b_or = train_perceptron(X, OR)
print("\nOR Gate")
print("Weights:", w_or)
print("Bias:", b_or)
print("Output:", test_perceptron(X, w_or, b_or))

w_xor, b_xor = train_perceptron(X, XOR)
print("\nXOR Gate")
print("Weights:", w_xor)
print("Bias:", b_xor)
print("Output:", test_perceptron(X, w_xor, b_xor))
```

AND

```
import numpy as np

def train_perceptron(X, y, epochs=10, learning_rate=0.1):
    weights = np.zeros(X.shape[1])
    bias = 0

    for epoch in range(epochs):
        for i in range(len(X)):
            z = np.dot(X[i], weights) + bias
            prediction = 1 if z >= 0 else 0
            error = y[i] - prediction

            weights = weights + learning_rate * error * X[i]
            bias = bias + learning_rate * error

    return weights, bias

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

AND = np.array([0, 0, 0, 1])

def test_perceptron(X, weights, bias):
    predictions = []
```

```
    for x in X:
        z = np.dot(x, weights) + bias
        prediction = 1 if z >= 0 else 0
        predictions.append(prediction)

    return predictions
w_and, b_and = train_perceptron(X, AND)
print("AND Gate")
print("Weights:", w_and)
print("Bias:", b_and)
print("Output:", test_perceptron(X, w_and, b_and))
```

OR

```
import numpy as np

def train_perceptron(X, y, epochs=10, learning_rate=0.1):
    weights = np.zeros(X.shape[1])
    bias = 0

    for epoch in range(epochs):
        for i in range(len(X)):
            z = np.dot(X[i], weights) + bias
            prediction = 1 if z >= 0 else 0
            error = y[i] - prediction

            weights = weights + learning_rate * error * X[i]
            bias = bias + learning_rate * error

    return weights, bias
```

```
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

OR = np.array([0, 1, 1, 1])

def test_perceptron(X, weights, bias):
    predictions = []

    for x in X:
        z = np.dot(x, weights) + bias
        prediction = 1 if z >= 0 else 0
        predictions.append(prediction)

    return predictions

w_or, b_or = train_perceptron(X, OR)

print("OR Gate")
print("Weights:", w_or)
print("Bias:", b_or)
print("Output:", test_perceptron(X, w_or, b_or))
```

XOR

```
import numpy as np
```

```
def train_perceptron(X, y, epochs=10, learning_rate=0.1):
    weights = np.zeros(X.shape[1])
    bias = 0

    for epoch in range(epochs):
        for i in range(len(X)):
            z = np.dot(X[i], weights) + bias
            prediction = 1 if z >= 0 else 0
            error = y[i] - prediction

            weights = weights + learning_rate * error * X[i]
            bias = bias + learning_rate * error

    return weights, bias

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

XOR = np.array([0, 1, 1, 0])

def test_perceptron(X, weights, bias):
    predictions = []

    for x in X:
        z = np.dot(x, weights) + bias
        prediction = 1 if z >= 0 else 0
```

```
        predictions.append(prediction)

    return predictions

w_xor, b_xor = train_perceptron(X, XOR)
print("XOR Gate")
print("Weights:", w_xor)
print("Bias:", b_xor)
print("Output:", test_perceptron(X, w_xor, b_xor))
```

Q) DELTA HEBBIAN

```
import numpy as np
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
T = np.array([0, 0, 0, 1])
eta = 0.1
weights = np.zeros(2)
bias = 0
epochs = 20
print("--- Training Delta Learning Rule (AND Gate) ---")
for epoch in range(epochs):
    total_error = 0
    for x, t in zip(X, T):
        y = np.dot(weights, x) + bias
        error = t - y
        weights = weights + eta * error * x
```

```
        bias = bias + eta * error
        total_error += error ** 2
    print(f"Epoch {epoch + 1}: MSE = {total_error / len(X):.4f}")
print("\nFinal Weights:", weights)
print("Final Bias:", bias)
print("\nPredictions:")
for x in X:
    y = np.dot(weights, x) + bias
    print(x, "->", round(y, 3))
print("\n--- Training Hebbian Learning Rule (OR Gate) ---")
X_hebb = np.array([
    [-1, -1],
    [-1,  1],
    [ 1, -1],
    [ 1,  1]
])
T_hebb = np.array([-1, 1, 1, 1])
eta_hebb = 1
weights_hebb = np.zeros(2)
bias_hebb = 0
print("Initial Weights:", weights_hebb)
print("Initial Bias:", bias_hebb)
for x, t in zip(X_hebb, T_hebb):
    net = np.dot(weights_hebb, x) + bias_hebb
    if net >= 0:
        y = 1
    else:
        y = -1
    weights_hebb = weights_hebb + eta_hebb * x * t
```

```
    bias_hebb = bias_hebb + eta_hebb * t
    print("\nInput:", x)
    print("Target:", t)
    print("Output:", y)
    print("Updated Weights:", weights_hebb)
    print("Updated Bias:", bias_hebb)
print("\nFinal Weights:", weights_hebb)
print("Final Bias:", bias_hebb)
print("\nTesting:")
for x in X_hebb:
    net = np.dot(weights_hebb, x) + bias_hebb
    if net >= 0:
        y = 1
    else:
        y = -1
    print(x, "->", y)
```

----------DELTA----------

```
import numpy as np
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])
T = np.array([0, 0, 0, 1])
eta = 0.1
weights = np.zeros(2)
bias = 0
epochs = 20
for epoch in range(epochs):
```

```
    total_error = 0
    for x, t in zip(X, T):
        # Calculate actual output
        y = np.dot(weights, x) + bias
        error = t - y
        weights = weights + eta * error * x
        bias = bias + eta * error
        total_error += error ** 2
    print(f"Epoch {epoch + 1}: MSE = {total_error / len(X):.4f}")
print("\nFinal Weights:", weights)
print("Final Bias:", bias)
print("\nPredictions:")
for x in X:
    y = np.dot(weights, x) + bias
    print(x, "->", round(y, 3))
```

----------HEBBIAN----------

```
import numpy as np
X = np.array([
    [-1, -1],
    [-1,  1],
    [ 1, -1],
    [ 1,  1]
])
T = np.array([-1, 1, 1, 1])
eta = 1
weights = np.zeros(2)
bias = 0
print("Initial Weights:", weights)
print("Initial Bias:", bias)
```

```
for x, t in zip(X, T):
    net = np.dot(weights, x) + bias
    if net >= 0:
        y = 1
    else:
        y = -1
    weights = weights + eta * x * t
    bias = bias + eta * t
    print("\nInput:", x)
    print("Target:", t)
    print("Output:", y)
    print("Updated Weights:", weights)
    print("Updated Bias:", bias)
print("\nFinal Weights:", weights)
print("Final Bias:", bias)
print("\nTesting:")
for x in X:
    net = np.dot(weights, x) + bias
    if net >= 0:
        y = 1
    else:
        y = -1
    print(x, "->", y)
```

Q) Mamdani FIS

```
import numpy as np
def triangular_membership(x, a, b, c):
    if x <= a or x >= c:
```

```
        return 0.0
    if x == b:
        return 1.0
    if x < b:
        return (x - a) / (b - a)
    return (c - x) / (c - b)
def low_temperature(temp):
    if temp <= 0:
        return 1.0
    elif temp >= 20:
        return 0.0
    else:
        return (20 - temp) / 20
def medium_temperature(temp):
    return triangular_membership(temp, 10, 20, 30)
def high_temperature(temp):
    if temp <= 20:
        return 0.0
    elif temp >= 40:
        return 1.0
    else:
        return (temp - 20) / 20
def slow_fan(speed):
    if speed <= 0:
        return 1.0
    elif speed >= 50:
        return 0.0
    else:
        return (50 - speed) / 50
def moderate_fan(speed):
    return triangular_membership(speed, 25, 50, 75)
```

```
def fast_fan(speed):
    if speed <= 50:
        return 0.0
    elif speed >= 100:
        return 1.0
    else:
        return (speed - 50) / 50
def fuzzy_fan_control(temperature):
    low = low_temperature(temperature)
    medium = medium_temperature(temperature)
    high = high_temperature(temperature)
    rule_slow = low
    rule_moderate = medium
    rule_fast = high
    speeds = np.arange(0, 101, 1)
    slow_output = np.array([min(rule_slow, slow_fan(speed)) for speed in speeds])
    moderate_output = np.array([min(rule_moderate, moderate_fan(speed)) for speed in speeds])
    fast_output = np.array([min(rule_fast, fast_fan(speed)) for speed in speeds])
    aggregated_output = np.maximum.reduce([slow_output, moderate_output, fast_output])
    denominator = np.sum(aggregated_output)
    if denominator == 0:
        return 0, low, medium, high
    fan_speed = np.sum(speeds * aggregated_output) / denominator
    return fan_speed, low, medium, high
temperature = 47
fan_speed, low, medium, high = fuzzy_fan_control(temperature)
print("=" * 50)
print("FUZZY LOGIC FAN SPEED CONTROL")
print("=" * 50)
print("\nInput Temperature:", temperature, "°C")
print("\nTemperature Membership Values:")
```

```
print("Low    :", round(low, 3))
print("Medium :", round(medium, 3))
print("High   :", round(high, 3))
print("\nFuzzy Rules:")
print("Rule 1: IF temperature is LOW THEN fan is SLOW")
print("Rule 2: IF temperature is MEDIUM THEN fan is MODERATE")
print("Rule 3: IF temperature is HIGH THEN fan is FAST")
print("\nFinal Output:")
print("Fan Speed =", round(fan_speed, 2), "%")
print("=" * 50)
```

Q) Write what is Prolog?

Prolog (PROgramming in LOGic) is a declarative logic-based programming language used mainly in AI, expert systems, and natural language processing. Unlike imperative languages (Python, Java, C) where you write step-by-step instructions, Prolog lets you define facts and rules, then uses an inference engine with backtracking to answer queries.

Core Components:

- Facts: Statements that are unconditionally true. (e.g., `parent(john, mary).`)
- Rules: Conditional statements using `:-` meaning "IF". (e.g., `grandparent(X, Y) :- parent(X, Z), parent(Z, Y).`)
- Queries: Questions asked to the system. (e.g., `?- grandparent(john, alice).`)

Answer to the query in the prompt: For `?- grandparent(john, alice).` with:

- `parent(john, mary).`
- `parent(mary, alice).`

Output: true (or Yes)

Primary Uses: Expert systems, NLP, chatbots, knowledge bases, theorem proving, and planning/scheduling.

Q) Uninformed search algorithm - BFS/DFS

Uninformed Search (Blind Search) is a class of search algorithms that explores a problem space using only the problem definition—without any domain-specific knowledge, heuristics, or estimates of how close a state is to the goal.

1. Breadth-First Search (BFS)

BFS explores the search tree level-by-level, expanding all nodes at a given depth before moving to nodes at the next depth level. It uses a FIFO Queue (First-In, First-Out) data structure.

- Characteristics:
    - Completeness: Complete (always finds a solution if one exists, provided the branching factor b is finite).
    - Optimality: Optimal for unweighted graphs (finds the solution with the minimum number of steps/edges).
    - Time Complexity: $O(b^d)$, where b is the branching factor and d is the depth of the shallowest solution.

    - Space Complexity: $O(b^d)$ (high memory usage because all nodes at the current level must be stored).

2. Depth-First Search (DFS)

DFS explores as deeply as possible along each branch before backtracking. When it hits a dead end or a goal node, it backtracks to the most recent node with unexplored paths. It uses a LIFO Stack (Last-In, First-Out) or recursion.

- Characteristics:
    - Completeness: Incomplete in infinite-depth spaces or graphs with loops (unless visited nodes are tracked or depth limits are set).
    - Optimality: Non-optimal (it may find a deep, sub-optimal solution before a shallow, optimal one).
    - Time Complexity: $O(b^m)$, where m is the maximum depth of the search space.
    - Space Complexity: $O(b \cdot m)$ (low memory usage because it only stores the current search path and unexplored siblings).

Q) Informed search - A* algorithm

Informed Search

Informed search leverages domain-specific knowledge via a heuristic function h(n), which estimates the remaining cost to reach the goal. This prioritizes promising paths and drastically reduces search effort.

A* Algorithm

A* evaluates nodes using:

f(n) = g(n) + h(n)

- g(n): Actual cost from the start node to node n.
- h(n): Estimated remaining cost from node n to the goal.
- f(n): Total estimated cost of path through node n.

A* expands the node with the lowest f(n) value. If h(n) is admissible (never overestimates the true remaining cost), A* is guaranteed to find the optimal path.

Example 1: Graph Pathfinding

- Goal: Find shortest path from P to T.
- Evaluation: Expands lowest f(n) nodes: P -> Q -> R -> S -> T
- Result: Total path cost = 8 (optimal path).

Example 2: 8-Puzzle Problem

- Goal: Slide tiles into blank space (0) to match the goal state using Manhattan Distance heuristic.

Initial State:

2 8 3

1 6 4

7 0 5

Goal State:

1 2 3

4 5 6

7 8 0

State Transitions: State 1:

1 2 3

4 0 6

7 5 8

State 2:

1 2 3

4 5 6

7 0 8

Goal State:

1 2 3

4 5 6

7 8 0

Q) Adversarial Search - MiniMax

Adversarial Search:

A search algorithm applied in game-playing and decision-making scenarios where multiple agents have conflicting goals. Termed "adversarial" because what benefits one player directly harms the opponent.

Real-World Examples: Chess, Tic-Tac-Toe, AI in video games.

Min-Max Algorithm:

A core framework for systematic decision-making in two-player, zero-sum games. It operates on the assumption that both participants play optimally:

MAX: Aims to maximize their advantage/score.

MIN: Aims to minimize MAX's advantage/score.

Working Mechanism:

Tree Construction: Builds a game tree representing all potential moves and counter-moves.

Evaluation: Leaf nodes receive numerical scores via a utility/evaluation function.

Backpropagation: Scores propagate upward:

MAX nodes select the maximum child value (best move for the player).

MIN nodes select the minimum child value (worst case for MAX, best for the opponent).

Decision: The root node's value yields the optimal guaranteed outcome, and the algorithm selects the move leading to the highest root child value.

Alpha-Beta Pruning: Prunes branches that cannot influence the final decision to speed up computation.

Execution: Repeats the process on every turn.

Conclusion:Successfully implemented adversarial search via the Min-Max algorithm and analyzed how it determines optimal moves under competitive conditions.

Q) NEURAL network using C programming

Theory & Concepts:
Neural Network: Brain-inspired model that learns patterns and decisions from data.
Components: Neurons (process I/O), Synapses (signal links), Weights (importance), Bias (threshold shift), Activation Function (firing rule, e.g., Step, Sigmoid, ReLU, Tanh), and Layers (Input, Hidden, Output).
Network Types: Single-layer Perceptron (linearly separable), MLP (non-linear), CNN (images), RNN (sequences).
Perceptron Implementation (AND Gate):
Inputs: x1, x2 | Parameters: w1, w2, bias (b)
Forward: z = (x1 * w1) + (x2 * w2) + b | Output: 1 if z >= 1.0, else 0
Delta Rule: error = target - output; update weights/bias by (learning_rate * error * input)
Workflow: Init params to 0 -> Train 10 epochs -> Test all 4 input pairs.
Conclusion:

Successfully solved the AND gate via single-layer perceptron, proving linear separability and core forward/update mechanisms (fails on non-linear cases like XOR).

Q).PECEPTRON

Theory:

Perceptron: Simplest artificial neural network (Frank Rosenblatt, 1958); a single-layer binary classifier calculating a weighted sum of inputs followed by an activation function.

Single-Layer Perceptron (SLP):

Structure: One input layer, one output layer (step activation function), no hidden layers.

Characteristics: Simple and fast; solves only linearly separable problems with a straight-line decision boundary.

Limitations: Cannot solve XOR; limited to simple classification tasks.

Visual: [x1, x2] + b -> y

Multi-Layer Perceptron (MLP):

Structure: Input layer, one or more hidden layers, output layer with non-linear activation functions.

Characteristics: Forms multiple decision boundaries; uses backpropagation; solves non-linearly separable problems like XOR.

Visual: [x1, x2] -> [h1, h2] + b -> y

Training and Testing Process:

Training (Perceptron Learning Rule):

Initialize weights and bias to 0.

For each sample:

Weighted sum: z = sum(w * x) + b

Prediction: 1 if z >= 0, else 0 (step function)

Error: error = target - prediction

Updates: w_new = w_old + learning_rate * error * x; b_new = b_old + learning_rate * error

Testing:

Feed test inputs into the trained model.

Compute predictions using final weights and bias.

Compare predictions against expected outputs.

Conclusion:

The single-layer perceptron successfully learns linearly separable gates (AND, OR) but fails on XOR. This confirms SLP is limited to linear problems, whereas non-linear tasks like XOR require multi-layer perceptrons with hidden layers.

Q) MAMDANI FIS

Theory:

The Mamdani FIS is one of the most widely used fuzzy inference methods. It uses fuzzy sets and rules to map inputs to outputs.

The process involves:

Fuzzification: Converting crisp inputs into fuzzy values using membership functions.

Rule Evaluation: Applying fuzzy rules to determine fuzzy outputs.

Aggregation: Combining fuzzy outputs from all rules.

Defuzzification: Converting fuzzy outputs back into crisp values.

Advantages:

Easy to interpret

Algorithm:

Define input and output variables.

Create membership functions for each variable.

Define fuzzy rules.

Fuzzify input values.

Apply rules and aggregate results.

Defuzzify to get crisp output.

Conclusion:

The Mamdani FIS successfully maps temperature to fan speed using fuzzy logic principles.

Q) HYBRID NEURO FIS

Example:

Heart Disease Risk Classification.

Inputs: Age, Blood Pressure, Cholesterol Level, Heart Rate.

Output: Heart Disease Risk (Low / Medium / High).

Theory:

Hybrid Neuro-Fuzzy System: Merges Artificial Neural Networks (ANN) with Fuzzy Logic.

Fuzzy Logic: Maps numerical patient data into linguistic terms (Low, Medium, High) using IF-THEN rules.

Neural Network Learning: Automatically tunes fuzzy parameters from training data (implemented via ANFIS).

Working:

Patient data is provided as input.

Inputs are fuzzified into linguistic values.

Fuzzy rule-base processes the inputs via IF-THEN rules.

ANFIS learning algorithm tunes fuzzy parameters using training data.

Fuzzy output is converted into a crisp value via defuzzification.

The system classifies risk as Low, Medium, or High.

Applications:

Fault detection, robot control, time-series prediction, industrial process control, medical diagnosis/risk classification, traffic predictions.

Conclusion:

Developed an ANFIS-based neuro-fuzzy system for heart disease risk classification. It unites neural network data-driven learning with interpretable, rule-based fuzzy reasoning.